

Studio: **come** costruire ciò che manca

Refactor abilitante, mappa di ownership anti-collisione tra Claude e codex, data-model, servizi, UI, test e task brief pronti all'uso. Dal **cosa** (analisi) al **come** (esecuzione).

- Fase 0 refactor
- Ownership per file
- 14 slice
- Migr 0021–0030
- Task brief codex

Claude · Agente A

Dati · Realtime · Piattaforma. Manager + Fase 0.
Migrazioni 0021–0025.

codex · Agente B

Composizione · Editor · Integrazione UI. Worktree isolata.
Migrazioni 0026–0030.

Studio — Piano di implementazione (esecuzione a due agenti)

Documento operativo per costruire ciò che manca alla sezione **Studio**, derivato da `docs/STUDIO_ANALISI_ROADMAP.pdf`. Qui non si discute *cosa* fare (già deciso nell'analisi), ma **come** farlo: file da toccare, data-model, servizi, UI, test, e soprattutto **come dividere il lavoro tra due agenti indipendenti (Claude + codex) senza collisioni**.

Progetto: **BNS Dashboard** (PRJ-2026-005) · Prodotto: BnsStudio Dashboard · Target release: **1.3.0** Base branch: `develop` · Migrazioni: **Claude 0021–0025, codex 0026–0030** (ultima in repo: `0020`).

0. Come leggere questo documento

1. **§1** definisce il modello di esecuzione a due agenti e le regole anti-collisione (leggere per primo).
2. **§2 (Fase 0)** è un refactor *bloccante*: va fatto e mergiato prima di qualsiasi lavoro parallelo.
3. **§3** è la mappa di ownership: chi possiede cosa. **§4** la sequenza a milestone.
4. **§5** contiene le specifiche dettagliate slice-per-slice (data-model + servizi + UI + test + DoD).
5. **§6** è il protocollo per i file condivisi. **§7** verifica e rilascio.
6. **Appendice A** contiene i *task brief* già pronti da consegnare a codex (copia-incolla).

Convenzione priorità (dall'analisi): **P0** parità indispensabile · **P1** alto valore · **P2** differenziante · **P3** opzionale.

1. Modello di esecuzione a due agenti

1.1 Ruoli

- **Claude = manager + agente A**. Assegna i task, possiede lo strato **dati / realtime / piattaforma**, esegue la Fase 0 (refactor), coordina i merge su `develop`.
- **codex = agente B (indipendente)**. Opera sul repo in una **worktree isolata**, possiede lo strato **composizione / editor / integrazione UI**. Riceve task self-contained (Appendice A) e li esegue in autonomia.

Un terzo esecutore indipendente è disponibile su richiesta: Claude può delegare uno *slice isolato* a un subagent (worktree separata) quando serve parallelismo extra; vale comunque la stessa mappa di ownership.

1.2 Regole anti-collisione (non negoziabili)

1. **Ownership per file**. Nessun agente modifica i file dell'altro. La mappa in §3 è la fonte di verità.
2. **File condivisi solo in append**. `apps/web/types/index.ts`, `apps/web/data/db.ts`, `apps/web/types/database.generated.ts`: solo **aggiunte**, in **regioni marcate** e distinte per agente (§6).
3. **Migrazioni per range prenotato**. Claude `0021–0025`, codex `0026–0030`. Mai riusare un numero. Una sola sequenza lineare: si applicano con `npx supabase db push` (lo fa l'utente; Claude verifica read-only via MCP, ref `twgdcmuxevaddfhjlfcn`).
4. **dist/ è gitignorato**. Mai `git add -A` alla cieca (cattura i symlink `node_modules` delle worktree). Aggiungere path specifici.
5. **Branch model**. Base `develop`. Ogni slice → `feature/studio-<slice>` → PR → merge su `develop`. **Rebase frequente su develop**. Release: `develop` → `main` + tag `v1.3.0`.
6. **Seam di integrazione**. I punti in cui il lavoro di un agente deve essere "montato" nella shell dell'altro (§2.3) sono **contratti espliciti** definiti in Fase 0: si toccano con micro-commit coordinati, mai in parallelo sullo stesso blocco.

1.3 Migration ledger (prenotazione)

#	CONTENUTO	AGENTE	SLICE
0021	<code>members</code> (presence_status, status_emoji, status_text, status_expires_at) + <code>studio_conversation_members</code> (notify_level, muted_until, is_favorite) + <code>studio_notification_prefs</code>	Claude	S1, S2
0022	<code>studio_message_pins</code> + <code>studio_conversation_bookmarks</code>	Claude	S3
0023	Full-text search: colonna <code>tsv</code> su <code>studio_messages</code> + indice GIN + trigger	Claude	S4
0024	<code>studio_scheduled_messages</code> + <code>studio_reminders</code>	Claude	S6b
0025	riserva Claude	Claude	—
0026	<code>studio_custom_emoji</code> + estensione reactions per emoji custom	codex	C3
0027	Eventi di business: <code>studio_messages.kind</code> (<code>user</code> <code>system</code> <code>event</code>) + <code>event_metadata</code> jsonb	codex	C6
0028	Link messaggio↔task: <code>tasks.metadata.sourceMessageId</code> (nessuna nuova tabella; documentare)	codex	C5
0029	riserva codex	codex	—
0030	riserva codex	codex	—

Ogni migrazione include le **RLS policy** coerenti con `0018_studio_rls_hardening` (isolamento per `organization_id`, helper `is_internal_org_member`).

2. Fase 0 — Refactor abilitante (BLOCCANTE) · owner: Claude

Perché. Oggi Studio è di fatto due file monolitici — `features/studio/StudioPage.tsx` (~1018 righe) e `services/studioService.ts` (~788 righe). Se due agenti li modificano in parallelo, le collisioni sono certe. La Fase 0 li spezza in moduli con confini netti, così ogni slice successivo tocca file distinti.

Regola: la Fase 0 è **un unico PR atomico** che va mergiato su `develop` **prima** di aprire qualsiasi slice. Dopo il merge, entrambi gli agenti fanno `rebase` su `develop`.

2.1 Struttura target

```
apps/web/features/studio/  
  StudioPage.tsx          # SHELL sottile: layout + composizione dei moduli (owner: codex post-  
Fase0)  
  studioContext.tsx       # provider: conversazione attiva, workspace, handler condivisi  
  components/  
    Sidebar.tsx           # lista canali/progetti/DM + viste speciali  
    ConversationHeader.tsx # titolo, azioni, slot per PinnedBar/Presence  
    MessageList.tsx       # scroller + paginazione (virtualizzabile)  
    MessageItem.tsx       # singolo messaggio + azioni (reaction/thread/save/pin)  
    ThreadPanel.tsx       # pannello thread  
  composer/  
    Composer.tsx          # orchestratore composizione  
    RichTextEditor.tsx    # editor + toolbar (C1)  
    MentionAutocomplete.tsx # popup menzioni inline (C2)  
    SlashCommandMenu.tsx  # command palette "/" (C5)  
  unfurl/EntityUnfurl.tsx # card entità "viva" (C4)  
  presence/PresenceIndicator.tsx, TypingIndicator.tsx # (S1)  
  pins/PinnedBar.tsx      # barra "in evidenza" (S3)  
  modals/ (ChannelModal, EditChannel, Members, Details, Notifications, NotificationPrefs...)  
  hooks/  
    useStudioRealtime.ts  # sottoscrizioni + update mirati cache (S5)  
    usePresence.ts        # Supabase Presence + typing (S1)  
    useStudioDraft.ts     # bozze persistenti per conversazione (C2)  
  lib/  
    studioMarkdown.ts     # formatMessage/readableMessageText estratti + estesi (C1)  
apps/web/services/  
  studioService.ts        # facade: ri-esporta i sotto-servizi (compat)  
  studio/  
    workspace.ts          # loadWorkspace, unread, reads  
    messages.ts           # list/send/update/delete/thread/search  
    reactions.ts, saves.ts  
    conversations.ts       # channel/dm/project CRUD, members  
    (nuovi in slice) presence.ts, notifications.ts, pins.ts, search.ts, scheduled.ts
```

2.2 Passi della Fase 0

1. Estrarre **senza cambiare comportamento** i componenti già presenti in `StudioPage.tsx` (`MessageItem`, `Composer`, `AttachmentPreview`, `formatMessage`, `readableMessageText`) nei file sopra.
2. Introdurre `studioContext.tsx` per non passare a mano `workspace` / `page` / `handler` ovunque.
3. Spezzare `studioService.ts` in `services/studio/*` mantenendo `studioService` come **facade** che ri-esporta (nessun call-site esterno deve rompersi).
4. Definire i **seam** (§2.3) come props/slot vuoti, così gli slice li riempiono senza toccare la shell.
5. Verde: `npm run typecheck && npm run lint && npm run test && npm run build`. Zero regressioni funzionali.

2.3 Seam di integrazione (contratti)

Punti dove uno slice deve "agganciarsi" alla shell. Definiti in Fase 0 come slot stabili:

SEAM	DOVE	CONTRATTO	RIEMPITO DA
HeaderSlots	ConversationHeader	presenceSlot? , pinnedBarSlot? , prefsButtonSlot?	Claude S1/S3/S2
MessageActions	MessageItem	callback onPin , onForward (oltre a react/save/edit/delete già presenti)	Claude S3
MessageBody	MessageItem	render pipeline renderContent() + renderUnfurls(refs)	codex C1/C4
ComposerPlugins	Composer	array di plugin (mention, slash, emoji) montati per config	codex C2/C3/C5
ListItemDecorator	MessageList	hook useTargetedCache() per update ottimistici	Claude S5

Il seam si tocca con **una riga di import + un elemento JSX**; questi micro-tocchi vanno in "integration commit" revisionati dal manager, mai editati in parallelo.

3. Mappa di ownership (slice → agente → file → migrazione)

Claude (agente A) — Dati · Realtime · Piattaforma

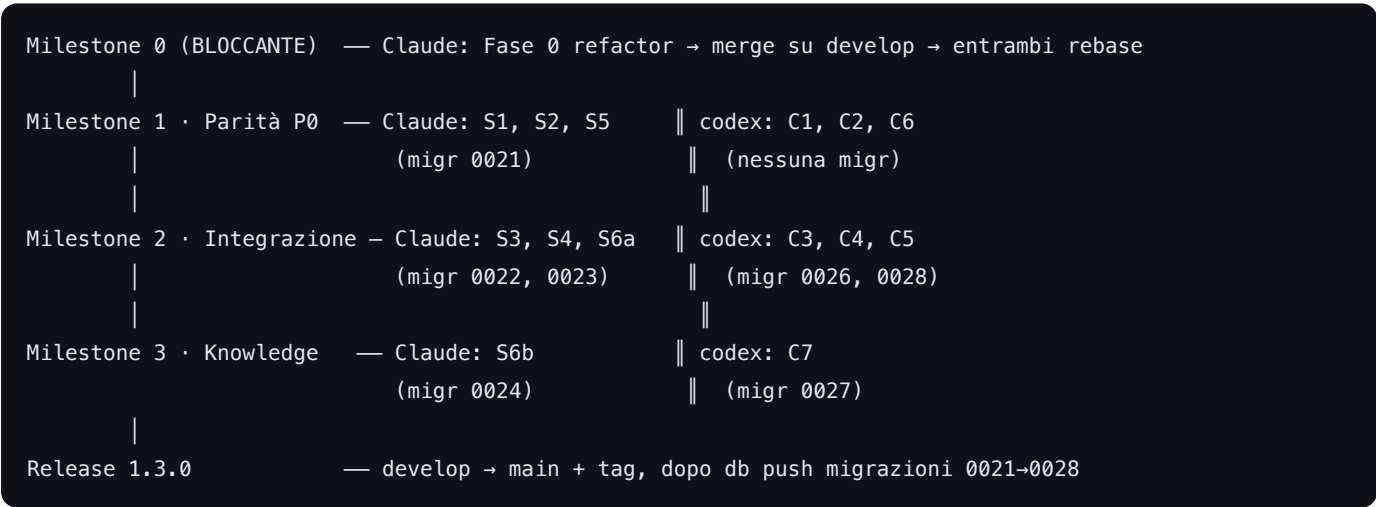
SLICE	TITOLO	FILE PRINCIPALI (OWNER CLAUDE)	MIGR	PRIO
S1	Presenza live + "sta scrivendo"	hooks/usePresence.ts , components/presence/* , services/studio/presence.ts	0021	P0
S2	Preferenze notifiche + mute	components/modals/NotificationPrefs.tsx , services/studio/notifications.ts	0021	P0
S3	Pin messaggi + segnalibri canale	components/pins/PinnedBar.tsx , services/studio/pins.ts	0022	P0
S4	Ricerca con filtri (from/in/has/before)	components/SearchPanel.tsx , services/studio/search.ts	0023	P0
S5	Realtime perf: update mirati + UI ottimistica	hooks/useStudioRealtime.ts , mutazioni in services/studio/messages.ts	—	P0
S6a	Push desktop (Tauri) + email digest	apps/desktop plugin notifiche, Edge Function studio-digest	—	P1
S6b	Messaggi programmati + promemoria	services/studio/scheduled.ts , components/composer/ScheduleMenu.tsx	0024	P2

codex (agente B) — Composizione · Editor · Integrazione UI

SLICE	TITOLO	FILE PRINCIPALI (OWNER CODEX)	MIGR	PRIO
C1	Editor ricco (liste/quote/codice/link/italico)	lib/studioMarkdown.ts , components/composer/RichTextEditor.tsx	—	P0
C2	Menzioni autocomplete inline + bozze persistenti	components/composer/MentionAutocomplete.tsx , hooks/useStudioDraft.ts	—	P0
C6	Mobile completo (drawer sidebar + thread fullscreen)	StudioPage.tsx (shell), components/Sidebar.tsx , components/ThreadPanel.tsx	—	P0
C3	Emoji picker completo + reazioni custom	components/ui/EmojiPicker.tsx , services/studio/reactions.ts (estensione)	0026	P1
C4	Unfurl live delle entità citate	components/unfurl/EntityUnfurl.tsx	—	P1
C5	Messaggio→Task + comandi rapidi /	components/composer/SlashCommandMenu.tsx , azioni in MessageItem menu	0028	P1
C7	Eventi di business nei canali di progetto	services/studio/events.ts , hook nei servizi finance	0027	P2

Zero sovrapposizioni di file salvo i seam (§2.3) e i file condivisi (§6). MessageItem.tsx è owner codex (composizione): l'azione "pin" di Claude S3 entra tramite il seam MessageActions (callback), non modificando il body.

4. Milestone & sequenza



Ordine consigliato dentro ogni milestone: prima le slice senza migrazione (sbloccano subito la UI), poi quelle con migrazione (richiedono db push dell'utente). Le slice di uno stesso agente sono seriali; tra i due agenti sono parallele.

5. Specifiche per slice

Ogni slice segue lo stesso schema: Obiettivo · Data-model · Servizio (demo+prod) · UI · Test · DoD · Dipendenze. Le firme SQL sono indicative: rispettare stile e RLS delle migrazioni 0016-0018 .

S1 · Presenza live + "sta scrivendo" — Claude — P0 — migr 0021

Obiettivo. Sostituire il pallino verde finto (member.status) con presenza reale e indicatore di digitazione.

Data-model (0021, parte members):

```

alter table members
  add column presence_status text,          -- online|away|dnd|offline (derivato lato client, opz.
persistito
  add column status_emoji text,
  add column status_text text,
  add column status_expires_at timestampz;

```

La **presenza in tempo reale** NON usa tabelle: canale **Supabase Presence** effimero per conversazione.

Servizio `services/studio/presence.ts` + `hooks/usePresence.ts` :

```

// canale effimero: nessuna scrittura DB, latenza minima
const ch = supabase.channel(`studio-presence:${conversationId}`, { config: { presence: { key: memberId
}}})
  .on('presence', { event: 'sync' }, () => setOnline(Object.keys(ch.presenceState())))
  .on('broadcast', { event: 'typing' }, ({ payload }) => pushTyping(payload.memberId));
await ch.track({ at: Date.now() });
// throttle del broadcast typing a ~1/2s mentre l'utente scrive

```

Demo (`IS_DEMO`): no-op che ritorna sempre "solo io online" (nessun realtime in Dexie).

UI: `PresenceIndicator` (avatar con dot verde live) montato via seam `HeaderSlots.presenceSlot` ;
`TypingIndicator` ("Mario sta scrivendo...") sopra il Composer.

Test: unit su throttle typing + su derivazione stato (online/away/expires). Presence realtime = smoke manuale.

DoD: aprendo una conversazione, gli altri membri connessi risultano online in <2s; il typing appare/scompare;
nessun rifetch DB generato dalla presenza. Demo non rompe.

Dipendenze: Fase 0 (seam `HeaderSlots`).

S2 · Preferenze notifiche + mute — Claude — P0 — migr 0021

Obiettivo. Controllo del rumore: livello notifiche per canale, mute temporaneo, preferiti, e preferenze globali (DND, keyword, digest).

Data-model (0021, parte prefs):

```

alter table studio_conversation_members
  add column notify_level text not null default 'all',    -- all|mentions|none
  add column muted_until timestampz,
  add column is_favorite boolean not null default false;

create table studio_notification_prefs (
  member_id uuid primary key references members(id) on delete cascade,
  organization_id uuid not null,
  dnd_start time, dnd_end time, timezone text,
  keywords text[] not null default '{}',
  email_digest text not null default 'daily',           -- off|daily|weekly
  created_at timestampz not null default now(),
  updated_at timestampz not null default now()
);
-- + RLS: member vede/scrive solo la propria riga; conv_members come 0018.

```

Servizio `services/studio/notifications.ts` : `getPrefs()`, `setChannelNotify(convId, level)`,
`muteChannel(convId, until)`, `toggleFavorite(convId)`, `setGlobalPrefs(...)`. Demo su Dexie (nuove tabelle

§6). La logica "devo notificare?" diventa: `notify_level` + `muted_until` + DND + match `keywords` .

UI: modale `NotificationPrefs` (globali) + voce per-canale nel menu header (seam `HeaderSlots.prefsButtonSlot`); canali mutati appaiono in grigio nella Sidebar (coordinare stile con codex C6 via prop, non editare Sidebar).

Test: unit sul predicato `shouldNotify(message, prefs, membership)` con casi: mention in canale muted, keyword, DND attivo, `none` .

DoD: un canale su "solo menzioni" non incrementa l'unread badge per messaggi non-menzione; mute nasconde i badge fino a scadenza; le preferenze persistono (demo+prod).

Dipendenze: Fase 0. Nota seam Sidebar (stile mutato) → coordinare con C6.

S3 · Pin messaggi + segnalibri di canale — Claude — P0 — migr 0022

Data-model (0022):

```
create table studio_message_pins (  
  id uuid primary key default gen_random_uuid(),  
  organization_id uuid not null,  
  conversation_id uuid not null references studio_conversations(id) on delete cascade,  
  message_id      uuid not null references studio_messages(id) on delete cascade,  
  pinned_by       uuid not null references members(id),  
  created_at timestamptz not null default now(),  
  unique (conversation_id, message_id)  
);  
  
create table studio_conversation_bookmarks (  
  id uuid primary key default gen_random_uuid(),  
  organization_id uuid not null,  
  conversation_id uuid not null references studio_conversations(id) on delete cascade,  
  title text not null, url text, entity_type text, entity_id uuid,  
  position int not null default 0,  
  created_at timestamptz not null default now()  
);  
  
-- RLS come i messaggi del canale (0018).
```

Servizio `services/studio/pins.ts` : `pin/unpin(messageId)` , `listPins(convId)` ,
`addBookmark/removeBookmark/reorder` .

UI: `PinnedBar` sotto l'header ("🔴 N in evidenza", pannello a comparsa) via seam `HeaderSlots.pinnedBarSlot` ; azione "Fissa/Rimuovi" nel menu del messaggio via seam `MessageActions.onPin` (codex aggiunge la voce, Claude la implementa).

Test: unit su unique pin (idempotenza), reorder bookmarks.

DoD: pin visibile a tutti i membri del canale, in cima; bookmark cliccabile apre URL o entità BNS; demo+prod.

Dipendenze: Fase 0; seam condiviso con codex su `MessageItem` (coordinato).

S4 · Ricerca con filtri — Claude — P0 — migr 0023

Obiettivo. Da "trova stringa" a ricerca vera con operatori `from:@membro in:#canale has:file before:2026-01-01` .

Data-model (0023):


```
alter table studio_messages add column tsv tsvector;
update studio_messages set tsv = to_tsvector('simple', coalesce(content, ''));
create index studio_messages_tsv_idx on studio_messages using gin(tsv);
create function studio_messages_tsv_trg() returns trigger as $$
begin new.tsv := to_tsvector('simple', coalesce(new.content, '')); return new; end $$ language plpgsql;
create trigger studio_messages_tsv before insert or update on studio_messages
for each row execute function studio_messages_tsv_trg();
```

Servizio `services/studio/search.ts` : parser degli operatori → query. Prod: `websearch_to_tsquery` + join filtri (`author_id`, `conversation_id`, presenza attachment, range date). Demo: filtro in-memory sui messaggi Dexie.

UI: `SearchPanel` con chip dei filtri attivi e risultati con **contesto** (autore, canale, snippet evidenziato); sostituisce il box di ricerca attuale nell'header (Claude possiede questo pezzo dell'header — coordinare in Fase 0).

Test: unit del parser (`from:@x has:file foo` → struct), e della query builder.

DoD: ricerca con almeno `from/in/has/before` funzionante e performante su archivio grande (indice GIN); demo+prod.

S5 · Realtime perf: update mirati + UI ottimistica — Claude — P0 — no migr

Obiettivo. Rimuovere l' `invalidateStudio()` globale (rifetch dell'intero workspace ad ogni evento) e le latenze percepite.

Interventi (`hooks/useStudioRealtime.ts` + mutazioni in `services/studio/messages.ts`):

- Sostituire `qc.invalidateQueries(['studio'])` con `qc.setQueryData` mirati per evento (`INSERT` messaggio → append alla pagina; `reaction / save` → patch del singolo record).
- **UI ottimistica** su invio, reazione, pin, save: aggiornare la cache prima della conferma, rollback on error.
- Deduplica messaggi già presenti (evita doppioni tra ottimistico e realtime, usare `id`).
- (Opz.) virtualizzazione `MessageList` per conversazioni lunghe.

Test: unit sul reducer di cache (append/patch/dedup) e sul rollback ottimistico.

DoD: inviando/reagendo, nessun rifetch di `loadWorkspace` ; la UI risponde <100ms; nessun messaggio duplicato con realtime attivo.

Dipendenze: Fase 0. **Nota:** questo slice cambia il modo in cui la shell consuma i dati → definire in Fase 0 il seam `ListItemDecorator / useTargetedCache` così codex (C1/C4) renderizza senza dipendere dai dettagli di cache.

S6a · Push desktop (Tauri) + email digest — Claude — P1

- **Desktop:** plugin notifiche Tauri per menzioni/DM quando la finestra non è a fuoco; badge sull'icona con conteggio non-letti. File in `apps/desktop` (owner Claude).
- **Email digest:** Edge Function `studio-digest` schedulata (cron Supabase) che invia un riepilogo dei non-letti/menzioni secondo `studio_notification_prefs.email_digest` . Deploy con `supabase functions deploy studio-digest` .

DoD: con app desktop in background, una menzione produce notifica nativa + badge; digest email inviata secondo preferenza.

S6b · Messaggi programmati + promemoria — Claude — P2 — migr 0024

```
create table studio_scheduled_messages ( id uuid pk, organization_id uuid, conversation_id uuid,
  author_id uuid, content text, metadata jsonb, send_at timestampz, sent_at timestampz, created_at
timestampz );
create table studio_reminders ( id uuid pk, organization_id uuid, member_id uuid, message_id uuid,
  remind_at timestampz, done_at timestampz, created_at timestampz );
```

Invio effettivo via Edge Function schedulata (o al caricamento client come fallback). UI: menu "Programma invio" nel Composer (seam `ComposerPlugins`) e "Ricordamelo" nel menu messaggio.

C1 · Editor ricco — codex — P0 — no migr

Obiettivo. Da markdown-lite (`**bold**`, ``code``) a editor completo, mantenendo il testo **grezzo** in `content`.

`lib/studioMarkdown.ts` (estrae e estende `formatMessage / readableMessageText` dalla Fase 0):

- Supportare: `*corsivo*`, `~barrato~`, titoli `#`, liste `- / 1.`, citazioni `>`, blocchi ````lang` con evidenziazione, link `[testo](url)` (oltre a mention/ref già gestiti).
- **Sanitizzare** l'output HTML: `dangerouslySetInnerHTML` deve ricevere solo tag whitelisted (riusare l'approccio anti-XSS del `MarkdownRenderer` già presente in `components/preview/`).

`components/composer/RichTextEditor.tsx`: textarea + toolbar leggera (bold/italic/code/link/list/quote) e scorciatoie `Cmd/Ctrl+B/I/K`. Nessun cambio di schema DB.

Test: unit su `studioMarkdown` per ogni costrutto + casi di sicurezza (script injection neutralizzato).

DoD: i costrutti si renderizzano correttamente e in sicurezza in messaggi, thread e anteprima ricerca.

Seam: `MessageBody.renderContent()` (Fase 0). Non tocca la logica dati.

C2 · Menzioni autocomplete inline + bozze persistenti — codex — P0

- **Autocomplete inline:** digitando `@`, popup che **filtra durante la digitazione** (oggi il menu non filtra); invio del token `@member:<id>` come già previsto. Estendere a `@canale / @qui` (distinti da `@tutti`).
`components/composer/MentionAutocomplete.tsx`.
- **Bozze persistenti:** `hooks/useStudioDraft.ts` salva la bozza per `conversationId` (localStorage in demo, opz. tabella in prod) e la ripristina cambiando canale.

Test: unit sul filtro menzioni e sul ciclo salva/ripristina bozza.

DoD: menzioni filtrate in tempo reale; la bozza sopravvive al cambio conversazione e al reload.

Seam: `ComposerPlugins` (Fase 0).

C6 · Mobile completo — codex — P0

Obiettivo. Studio usabile sotto `md`: oggi la Sidebar è `hidden ... md:flex` e il thread è `hidden ... lg:flex`.

- Sidebar in **drawer** (off-canvas) con toggle nell'header su mobile.
- ThreadPanel a **schermo intero** su mobile (route/overlay) invece di colonna laterale.
- Gestione focus/scroll e area sicura. `StudioPage.tsx` (shell), `Sidebar.tsx`, `ThreadPanel.tsx`.

Test: smoke responsive (preview `preview_resize` mobile) + unit su stato apertura drawer.

DoD: su viewport 375px si naviga tra conversazioni, si legge/scrive, si aprono i thread senza layout rotti.

C3 · Emoji picker completo + reazioni custom — codex — P1 — migr 0026

```
create table studio_custom_emoji ( id uuid pk, organization_id uuid, shortcode text, file_id uuid,
created_by uuid, created_at timestamptz, unique(organization_id, shortcode) );
-- reactions: consentire emoji = shortcode custom oltre a unicode
```

`components/ui/EmojiPicker.tsx` (riusabile altrove) + estensione `services/studio/reactions.ts`. Sostituisce le 6 emoji fisse.

DoD: picker completo con ricerca; reazioni unicode + custom dell'org; demo+prod.

C4 · Unfurl live delle entità — codex — P1 — no migr

Obiettivo. `[[ref:...]]` non più solo link: **card viva** con stato attuale.

`components/unfurl/EntityUnfurl.tsx` legge `metadata.references` e interroga i `repositories.*` per rendere:

- **invoice** → numero, importo, **stato pagamento** (badge live), scadenza
- **project** → codice, nome, avanzamento/health
- **estimate** → numero, importo, stato (approvato/in attesa)
- **task** → titolo, stato, assegnatario, scadenza
- **payment/contract/client/file** → sintesi coerente

Nessuna nuova tabella: si arricchisce il rendering. Cache via React Query per non rifetchare.

Test: unit sul mapping ref→card e sugli stati badge.

DoD: citando una fattura appare la card con lo stato reale; click apre l'entità; nessun N+1 (query batch/deduplicate).

Seam: `MessageBody.renderUnfurls(refs)` (Fase 0).

C5 · Messaggio→Task + comandi rapidi / — codex — P1 — migr 0028

- **Messaggio→Task:** voce nel menu "... " del messaggio → precompila `NewTask` (progetto del canale, assegnatario da menzione, scadenza), crea link bidirezionale `tasks.metadata.sourceMessageId` e posta un riferimento nel thread. Usa i `repositories` /hook task esistenti.
- **Command palette** / : estende il selettore entità attuale a **verbi**: `/task <titolo>`, `/preventivo <cliente>`, `/cerca <query>`, `/promemoria <quando>`. `components/composer/SlashCommandMenu.tsx`.

DoD: da un messaggio nasce un task collegato in <3 click; i comandi creano/trovano entità senza uscire dal Composer.

Seam: `ComposerPlugins` + menu `MessageItem` (callback, coordinato con Claude S3 sullo stesso menu).

C7 · Eventi di business nei canali — codex — P2 — migr 0027

```
alter table studio_messages
add column kind text not null default 'user', -- user|system|event
add column event_metadata jsonb;
```

Hook nei servizi finance/project: al verificarsi di eventi (fattura pagata, preventivo approvato, task in ritardo) si inserisce un messaggio `kind='event'` nel canale di progetto. Render dedicato (riga di sistema, non bubble utente).

DoD: pagando una fattura compare l'evento "FT-014 pagata ✓" nel canale del progetto collegato; gli eventi non generano notifiche rumorose (rispettano S2).

6. Protocollo file condivisi

6.1 `types/index.ts` (append-only, regioni per agente)

Aggiungere i nuovi tipi in blocchi marcati, **in fondo alla sezione Studio**, senza riordinare l'esistente:

```
// — Studio · Claude (S1-S6) —————
export interface StudioNotificationPrefs { ... }
export interface StudioMessagePin { ... }
export interface StudioConversationBookmark { ... }
export interface StudioScheduledMessage { ... }
// — Studio · codex (C1-C7) —————
export interface StudioCustomEmoji { ... }
export type StudioMessageKind = 'user' | 'system' | 'event';
```

6.2 `data/db.ts` (bump versione Dexie + tabelle demo)

DB `bns-studio-os`. Aggiungere le nuove tabelle in **una nuova** `version(n+1).stores({...})` (Dexie richiede solo i delta). Claude aggiunge le sue (`studioMessagePins`, `studioConversationBookmarks`, `studioNotificationPrefs`, `studioScheduledMessages`); codex le sue (`studioCustomEmoji`). **Coordinare il numero di versione** (chi mergia prima usa `version(k)`, il secondo `version(k+1)` in rebase) per non collidere sullo stesso bump.

6.3 `types/database.generated.ts`

Va tenuto allineato **a mano** dopo ogni migrazione applicata (o rigenerato con `mcp ... generate_typescript_types`). Chi introduce la migrazione aggiorna la propria porzione; append-only.

6.4 Shell `StudioPage.tsx`

Post-Fase 0 è **owner codex**. I componenti di Claude (Presence/PinnedBar/NotificationPrefs) entrano **solo** via i seam `HeaderSlots` come props: codex aggiunge import+elemento in un "integration commit" concordato. Nessun altro motivo per cui Claude debba editare `StudioPage.tsx`.

7. Verifica & rilascio

Gate per ogni PR (entrambi gli agenti):

```
npm run typecheck && npm run lint && npm run test && npm run build
```

- Ogni slice porta i **propri test** (`tests/unit/`). Nessun merge con test rossi.
- Verifica funzionale reale nel preview (dev su :5173) del flusso toccato — non fidarsi del solo verde.
- **Doppia modalità:** ogni nuova API di servizio deve funzionare in **demo (Dexie)** e **prod (Supabase)**.

Migrazioni: l'utente esegue `npm run supabase db push` dalla root nell'ordine `0021 → 0028`. Claude verifica read-only via MCP (ref `twgdcmuxevaddfhjlfcn`) prima e dopo. Branch Supabase non disponibile (piano free).

Release 1.3.0: allineare la versione nei 3 punti (`package.json`, `brandConfig`, `tauri.conf`), `develop → main`, tag `v1.3.0`. Convenzione commit `feat:/fix:`, corpo con `Co-Authored-By`.

Definition of Done complessiva: la checklist finale in `STUDIO_ANALISI_ROADMAP.pdf` passa da assente→presente per tutte le voci P0/P1; nessun rifetch globale; mobile usabile; demo+prod verdi.

7bis. Git — come lavora ogni agente (comandi)

Modello: `main` = produzione, `develop` = sviluppo, `feature/*` = feature. Base di ogni lavoro: `develop`. Il **manager (Claude)** possiede il merge su `develop`. Convenzione commit `feat:/fix:` con corpo `Co-Authored-By`. Remote: `origin` (GitHub `simonebonusoo/bns-studio-dashboard`).

Ordine assoluto (non saltare)

1. **Push dei documenti** su `develop` (o su un branch dedicato) — senza questo codex non li vede su GitHub.
2. **Claude apre e mergia la Fase 0** (`feature/studio-fase0` → PR → `develop`). Finché non è su `develop`, gli slice sono bloccati (i file/struttura non esistono).
3. Solo allora: i due agenti aprono gli slice in parallelo, ciascuno dal proprio range di file e migrazioni.

Corsia di codex (agente connesso a GitHub)

Per un codex connesso al repo GitHub l'isolamento è il **branch stesso** (non serve worktree locale):

```
git checkout develop && git pull --ff-only origin develop
git checkout -b feature/studio-c1-editor-ricco          # un branch per slice (C1..C7)
# ...lavoro solo sui file di ownership codex (§3) + append nelle regioni marcate (§6)...
npm run typecheck && npm run lint && npm run test && npm run build  # gate obbligatorio
git add apps/web/features/studio/lib/studioMarkdown.ts
apps/web/features/studio/components/composer/RichTextEditor.tsx tests/unit/...
                                                                    # MAI git add -A (cattura dist/ e symlink
node_modules)
git commit -m "feat(studio): editor ricco (C1)"          # + Co-Authored-By nel corpo
git push -u origin feature/studio-c1-editor-ricco
# aprire PR verso develop; assegnare la review al manager (Claude)
```

Prima di aprire la PR e ogni volta che `develop` avanza: **rebase**.

```
git fetch origin && git rebase origin/develop          # risolvi conflitti sui SOLI file condivisi
(append)
```

Branch naming codex: `feature/studio-c<n>-<slug>`. Migrazioni **solo** in `0026-0030`.

Corsia di Claude (locale, manager)

- Fase 0 e slice S* su branch `feature/studio-fase0`, `feature/studio-s<n>-<slug>`. Migrazioni **0021-0025**.
- Lavoro locale con **worktree isolate** quando serve parallelismo (aggiungere `/node_modules` e `/apps/web/node_modules` al `.gitignore` del branch per non committare i symlink della worktree).
- **Merge su `develop`**: il manager valida (gate verde + verifica funzionale), poi mergia le PR di entrambi in ordine di milestone, risolvendo eventuali conflitti sui file condivisi (§6). Release: `develop` → `main` + tag `v1.3.0`.

Regole git valide per entrambi

- Mai committare `dist/` (gitignorato) né i symlink `node_modules`; niente `git add -A`.
- Un branch = uno slice = una PR piccola e rivedibile. Rebase, non merge-commit, su `develop`.
- Toccare i **seam** (§2.3) solo tramite il contratto Fase 0, in "integration commit" concordati col manager.
- Le **migrazioni si applicano solo con `npm run supabase db push`** (lo fa l'utente): un agente le *scrive* nel proprio range, non le applica; Claude verifica read-only via MCP.

Appendice A — Task brief pronti per codex

Ogni brief è **self-contained**: contiene ownership, migrazione, seam e DoD. Consegnare **uno alla volta**, in ordine di milestone, dopo che la **Fase 0 è mergiata su `develop`**. codex lavora in **worktree isolata**, branch `feature/studio-<slice>`, PR su `develop`, rebase frequente. Ranges migrazione codex: **0026-0030**.

A0 — Prerequisito. «Rebase su `develop` (contiene il refactor Fase 0 di Studio: shell in `features/studio/`, servizi in `services/studio/*`, seam definiti). Non modificare file fuori dalla tua ownership (vedi `docs/STUDIO_IMPLEMENTATION_PLAN.md` §3). File condivisi solo in append (§6).»

A1 — C1 Editor ricco (P0). «Implementa l'editor ricco di Studio. Owner:

`apps/web/features/studio/lib/studioMarkdown.ts`, `components/composer/RichTextEditor.tsx`. Estendi `formatMessage` a corsivo/barrato/titoli/liste/quote/blocchi codice/link, mantenendo il testo grezzo in `content`; **sanitizza** l'HTML (riusa l'anti-XSS di `components/preview/MarkdownRenderer`). Aggancio via seam `MessageBody.renderContent()`. Test unit per ogni costrutto + injection. Nessuna migrazione. DoD: costrutti resi in sicurezza in messaggi/thread/ricerca. Gate: typecheck+lint+test+build.»

A2 — C2 Menzioni + bozze (P0). «Autocomplete menzioni inline che filtra durante la digitazione (`components/composer/MentionAutocomplete.tsx`), token `@member:<id>`, aggiungi `@canale / @qui`. Bozze persistenti per `conversationId` (`hooks/useStudioDraft.ts`, `localStorage` in demo). Seam `ComposerPlugins`. Test su filtro e salva/ripristina. DoD: menzioni filtrate live, bozza sopravvive a cambio canale/reload.»

A3 — C6 Mobile (P0). «Rendi Studio usabile sotto `md`: Sidebar in drawer off-canvas con toggle; ThreadPanel a schermo intero su mobile. Owner: `StudioPage.tsx` (shell), `Sidebar.tsx`, `ThreadPanel.tsx`. Non toccare i servizi. Coordina con Claude S2 lo stile dei canali mutati (arriva come prop). Verifica responsive 375px nel preview. DoD: navigazione/lettura/scrittura/thread ok su mobile.»

A4 — C3 Emoji picker + custom (P1, migr 0026). «`components/ui/EmojiPicker.tsx` completo con ricerca; migrazione **0026** `studio_custom_emoji` + estensione reazioni custom in `services/studio/reactions.ts` (con RLS come 0018). Aggiorna la tua porzione di `database.generated.ts`. DoD: reazioni unicode+custom, demo+prod.»

A5 — C4 Unfurl live (P1). «`components/unfurl/EntityUnfurl.tsx`: card viva per `metadata.references` (invoice→stato pagamento, project→avanzamento, estimate→stato, task→stato/assegnatario, ecc.) leggendo i `repositories.*`, cache React Query, nessun N+1. Seam `MessageBody.renderUnfurls`. Test mapping ref→card. Nessuna migrazione.»

A6 — C5 Messaggio→Task + / (P1, migr 0028). «Azione "Crea task" nel menu messaggio (precompila `NewTask`, link `tasks.metadata.sourceMessageId`); command palette `/` con `/task /preventivo /cerca /promemoria` (`SlashCommandMenu.tsx`). Coordina con Claude S3 sul menu `MessageItem` (callback, non riscrivere il menu). Documenta lo schema link in migr **0028**. DoD: task da messaggio in <3 click; comandi operativi.»

A7 — C7 Eventi business (P2, migr 0027). «Migr **0027**: `studio_messages.kind` + `event_metadata`. Hook nei servizi finance/project per postare eventi (`kind='event'`) nel canale di progetto; render riga di sistema; rispetta le preferenze notifiche (S2). DoD: fattura pagata → evento nel canale collegato, senza rumore.»

Appendice B — Checklist di coordinamento per ogni PR

- ☐ Rebasato su `develop` aggiornato; Fase 0 presente.
- ☐ Solo file di mia ownership (§3); file condivisi solo in append nelle regioni marcate (§6).
- ☐ Migrazione nel mio range (Claude 0021–0025 · codex 0026–0030), con RLS coerente a 0018.
- ☐ Servizio funzionante in **demo** e **prod**; tabelle Dexie aggiunte con bump versione coordinato.
- ☐ Test unit dello slice presenti e verdi; `typecheck+lint+test+build` verdi.
- ☐ Nessun `git add -A` (path specifici; mai `dist/` né symlink `node_modules`).
- ☐ Verifica funzionale nel preview del flusso toccato.
- ☐ Seam toccati solo tramite il contratto Fase 0, in integration commit concordati.

Documento derivato da `docs/STUDIO_ANALISI_ROADMAP.pdf`. Le stime e le firme SQL sono indicative e vanno validate in fase di planning/PR. Aggiornare questo file se la mappa di ownership cambia.